

Pilot Live Companion, Screen Understanding, Provider Evidence, and Observer Mode

AION Device Fabric Technical Supplement

Tessaris / AION Product and Technical Record

29 August 2026

Fabric implementation versions 0.15.0–0.17.0

Abstract

This supplement documents the implemented transition from television control to evidence-grounded television understanding in AION Device Fabric. Fabric 0.15 introduced Pilot Live Companion and signed, rights-aware Pilot Moments. Fabric 0.16 added local phone-camera perception, positioned optical character recognition (OCR), structured screen fusion, confidence-labelled findings, and private owner correction. Fabric 0.17 added provider-reference normalization, optional official YouTube metadata enrichment, and a short-lived explicit Observer Mode authority. Together these capabilities allow Pilot to reason about recent television speech and owner-supplied visual evidence without claiming unsupported access to protected programme pixels, copying protected media, inferring a person’s identity from a face, or treating provider metadata as proof of subscription entitlement. This document records the architecture, schemas, execution flows, trust boundaries, verification results, and remaining production work.

1 Scope and Implemented Status

The work described here is additive to the existing mother-brain, signed capability, webOS gateway, private-phone, audit-ledger, and television-Canvas architecture. It does not replace COMDEX intelligence files and does not grant the television direct access to COMDEX memory, service credentials, device pairing secrets, or private household identity data.

Table 1: Implemented releases covered by this supplement.

Release	Principal object	Capability introduced	Verification
0.15.0	<code>pilot.live.context.v1</code> and <code>pilot.moment.v1</code>	Explicit live fact-check, explanation, and rights-aware Moment preparation using recent memory-only audio and permitted TV state	136 tests; real LG state observation and Moment preparation
0.16.0	<code>pilot.screen-understanding.v1</code>	Local Apple Vision OCR geometry, screen-element detection, source/confidence fusion, and persona-bound correction	144 tests; real OCR geometry processing; no live TV commands
0.17.0	<code>pilot.provider-metadata.v1</code> and <code>pilot.explicit-observer.v1</code>	Provider identifiers, optional official YouTube enrichment, and explicit short-lived camera-session leases	149 tests; audit chain valid; no live TV commands

The user-facing commands introduced by this programme include:

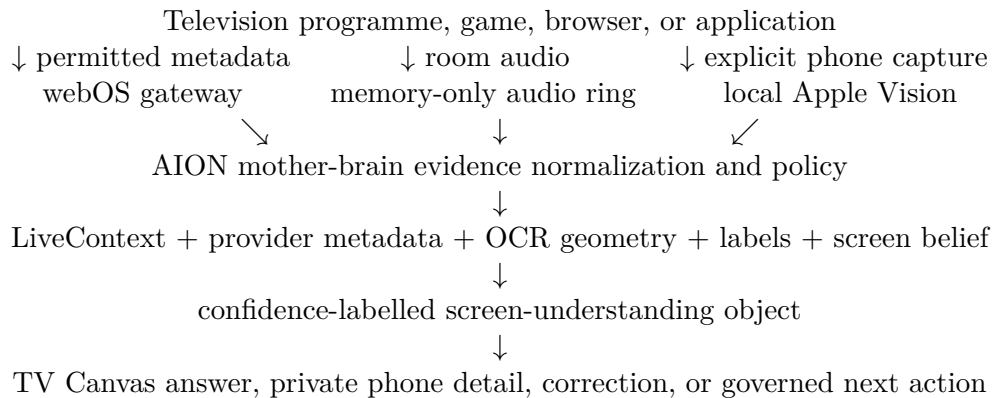
- “Pilot, fact check that.”
- “Pilot, explain that.”
- “Pilot, share that moment.”
- “Pilot, what is on the screen?”
- “Pilot, what is the score?”
- “Pilot, read the subtitle.”
- “Pilot, what product is that?”

These phrases are entry points into governed intents rather than authority to perform arbitrary external actions. A request may produce an answer, a prepared context object, an explicit request for more evidence, or an honest refusal. It does not silently purchase, message, publish, identify a person, or export protected media.

2 System Architecture

2.1 Evidence flow

The implemented data path is:



The restricted television remains a gateway-managed surface. The mother node performs planning, canonicalization, signing, local perception, policy checks, evidence retrieval, and auditing. The private phone is both a control surface and an optional owner-visible sensor. The TV Canvas receives only a sanitized projection.

2.2 Evidence classes

Pilot distinguishes rather than silently merges its inputs:

1. **Device evidence:** permitted webOS foreground-application, audio, playback, and cached state.
2. **Ephemeral transcript evidence:** locally transcribed material from the bounded recent-audio buffer, used only after an explicit request.
3. **Owner-captured visual evidence:** OCR observations, bounding boxes, classifications, and an image digest derived from an explicit phone submission.
4. **Provider evidence:** normalized official URLs or identifiers and, where configured, bounded official API metadata.

5. **Prior screen belief:** the most recent confidence-labelled Fabric view of the active surface.
6. **Owner correction:** a persona-bound statement correcting one supported field of the current observation.

Every fusion result records source descriptions and confidence values. A source conflict is represented explicitly. Missing evidence is not converted into fabricated certainty.

3 Fabric 0.15: Pilot Live Companion

3.1 Memory-only recent-audio window

The voice service maintains a bounded in-memory ring containing at most approximately 30 seconds of recent audio. In the current implementation the ring is a fixed-length deque sampled in bounded blocks. It is cleared when the microphone sleeps and when the voice process stops. Raw audio is not written to the Fabric runtime directory, capability ledger, Canvas, phone interface, or Moment object.

The ring is not itself an authorization to retain or research television speech. Persistence begins only when the user makes an explicit contextual request. The mother then constructs a small structured LiveContext from bounded recent transcript fragments and freshly observed permitted TV state.

3.2 LiveContext schema

The persisted object has schema identifier `pilot.live.context.v1`. Its principal fields are:

Field	Meaning
<code>created_at</code>	UTC creation time.
<code>surface</code>	Current confidence-labelled Fabric surface.
<code>app</code>	Bounded foreground application identifier and title.
<code>playback</code>	Exposed title, state, position, or duration fields when available.
<code>recent_statement</code>	Most recent bounded, non-wake transcript fragment.
<code>recent_transcripts</code>	At most four normalized transcript fragments.
<code>evidence</code>	Typed descriptions of webOS and ephemeral transcript sources.
<code>confidence</code>	Context confidence, not a truth score for the statement.
<code>privacy</code>	Raw-audio retention and explicit-request declarations.
<code>context_hash</code>	Canonical hash of the structured record.

The context hash is calculated over canonical bytes. It permits later audit records and Moment objects to refer to the exact evidence object without embedding raw audio.

3.3 Fact-check and explanation flow

For “fact check that” or “explain that”, the runtime performs the following bounded sequence:

1. Verify the signed capability corresponding to the requested intent.
2. Observe fresh permitted webOS state and record its action receipt.
3. Extract a bounded recent statement from the memory-only transcript window; if unavailable, use a sufficiently supported captured subtitle or owner correction.
4. Construct and hash the LiveContext.
5. Route the question to the research layer with the appropriate fact-check or explanation mode.

6. Prefer current, attributable evidence and retain sanitized results rather than the raw provider response.
7. Present a compact result on the TV Canvas and fuller evidence on the private phone.
8. Write an audit event containing the intent, evidence hashes, and result state.

A fact-check result is evidence-labelled. Where research evidence is not available, Pilot must not invent a verdict; it returns an insufficient-evidence state or an explicit search handoff. The system therefore distinguishes *what was said*, *what the TV reports is active*, and *what external evidence supports*.

4 Fabric 0.15: Signed Pilot Moments

4.1 Purpose and rights boundary

“Share that moment” currently prepares a portable context object rather than copying television media. The object uses schema `pilot.moment.v1`. It may contain application and playback metadata, a bounded recent statement, the originating context hash, a requested interval of at most 30 seconds, and an optional recipient hint. It explicitly records:

- `protected_audio_copied = false`;
- `protected_video_copied = false`;
- provider-native sharing is preferred; and
- a fallback card contains context rather than programme media.

The current provider-aware hierarchy is:

1. YouTube-like surfaces may prepare a provider timestamp link.
2. Twitch-like surfaces may prepare a provider-native clip operation, pending an authorized provider adapter.
3. Other surfaces produce a signed context card unless an official native sharing mechanism is exposed.

The Moment remains `prepared_not_sent`. Selecting a recipient and sending through a messaging service remain separate private actions.

4.2 Integrity model

Let M be the canonical Moment payload. Fabric calculates

$$h_M = \text{SHA256}(\text{canonical}(M)), \quad (1)$$

and signs the canonical bytes with the issuing node identity:

$$\sigma_M = \text{Sign}_{K_{\text{issuer}}}(\text{canonical}(M)). \quad (2)$$

The stored record includes M , h_M , the issuer public key, and σ_M . This establishes provenance and tamper evidence; it does not create media rights or prove that a recipient is entitled to play the referenced programme.

5 Fabric 0.16: Non-Disruptive Screen Understanding

5.1 Owner-visible phone perception

Because LG webOS does not expose a supported general-purpose third-party screen-capture interface, the implemented visual path uses a photograph submitted explicitly from the private phone.[5] The service accepts JPEG, PNG, or HEIC content up to 5 MiB. It writes a permission-restricted temporary file, runs local Apple Vision, and deletes the temporary image in a **finally** cleanup path.

The persisted `aion.phone.visual-perception.v1` record contains only:

- SHA-256 digest and byte count of the submitted image;
- explicit-capture source and consent labels;
- at most 40 normalized OCR strings;
- at most 40 positioned OCR observations;
- at most 12 bounded image-classification labels;
- inferred surface, view, cues, confidence, and timestamp; and
- `image_retained = false`.

Each positioned OCR item contains bounded normalized coordinates $x, y, w, h \in [0, 1]$ and recognition confidence. Geometry allows Pilot to distinguish likely subtitle regions from menus and other screen text.

5.2 Fusion model

The screen fusion service combines phone perception P , permitted device state D , explicit live context L , previous screen belief B , and provider metadata R :

$$S = F(P, D, L, B, R). \quad (3)$$

The current implementation is a bounded deterministic evidence classifier, not an unrestricted claim of visual understanding. For each supported finding j , it emits detection state d_j , confidence c_j , supporting source IDs, and a normalized evidence subset. The overall confidence is the mean of detected supported-element confidences, or the base perception confidence when no supported element is detected:

$$C(S) = \begin{cases} |J|^{-1} \sum_{j \in J} c_j, & |J| > 0, \\ c_P, & |J| = 0. \end{cases} \quad (4)$$

This value indicates evidence strength within the implemented detector set; it is not a probability that every semantic statement about the scene is true.

5.3 Implemented detectors

The resulting `pilot.screen-understanding.v1` object contains an observation ID, app and surface, summary, confidence, source-conflict flag, typed source records, supported findings, provider metadata, corrections, privacy declarations, and canonical evidence hash.

Table 2: Current screen-understanding findings.

Finding	Evidence used	Important boundary
Subtitles	OCR position, centre alignment, line length, and recognition confidence	Likely subtitle text, not guaranteed programme transcript
Scoreboard	Team/score pattern, match clock, sports terms, and image labels	Does not independently verify the official match result
Products	Currency/price expressions, purchase language, and product labels	Any save, basket, or purchase remains private-confirmation-only
Locations	Map, route, distance, station, airport, landmark, and city cues	Does not infer a private location or authorize navigation
Game HUD	Game surface state plus mission, level, score, health, or quest text	Does not control gameplay or infer hidden game state
People	Generic person-related image labels	Faces remain unknown; identity is never guessed

5.4 Private correction

The phone controller lets the owner correct only the latest observation and only an allowlisted field: subtitle, scoreboard, product, location, or scene. The correction is normalized, bounded to 240 characters, attached to a private persona, timestamped, and incorporated into a new evidence hash. A stale observation ID is rejected. Person identity is intentionally absent from the allowlist.

Corrections are evidence supplied by the owner, not automatic ground truth. They can support later explanations or fact-check requests, but should remain source-labelled in any downstream result.

6 Fabric 0.17: Provider Evidence

6.1 Normalized provider record

The `ProviderMetadataResolver` accepts a bounded set of HTTPS references obtained from the current evidence context. It recognizes supported YouTube video IDs, Netflix title IDs, and Twitch clip IDs and writes a `pilot.provider-metadata.v1` object. Common fields include provider, content ID, canonical official URL, metadata status, confidence, source links, limitations, application evidence, entitlement flags, and evidence hash.

The following invariants always apply:

- `availability_verified = false` unless a future authorized availability adapter proves otherwise;
- `account_entitlement_verified = false` unless a future persona-specific account adapter proves otherwise;
- metadata is not treated as permission to play, copy, clip, or send protected content; and
- an unavailable provider API cannot prevent local screen perception.

6.2 YouTube enrichment

Without a configured API key, a recognized YouTube URL produces a `provider_identifier_only` record. When the owner configures `YOUTUBE_API_KEY`, the mother may call the official YouTube Data API `videos.list` endpoint using a restricted field projection. The response is limited to 500,000 bytes and may enrich the record with title, channel, publication time, ISO-8601 duration, caption availability, live state, and embeddability.[1] A timeout, invalid key, oversized response, invalid JSON, or other provider failure produces `official_api_unavailable` while preserving the identifier-only evidence path.

6.3 Netflix and Twitch boundaries

A Netflix title reference records the official title ID and identifies Netflix’s supported mobile Moments feature as a potential native-share route.[2] It does not claim that Moments can be driven from the LG browser, that a title is available in the household’s region, or that Pilot may copy the programme.

A Twitch clip reference records the official clip identifier and a native-clip boundary. Enriched Twitch operations remain unavailable until a developer application and user-authorized adapter are connected. Twitch embedded video also imposes HTTPS and parent-domain requirements that a production surface must satisfy.[3]

7 Fabric 0.17: Explicit Observer Mode

7.1 Authority and session lifecycle

Observer Mode is a backend and phone-interface foundation for repeated visual sampling. It is deliberately not a hidden camera mode. The owner must press *Start Observer Mode* on the private phone and grant visible browser or native camera permission.

Starting a session creates:

- a random session identifier;
- a 256-bit URL-safe bearer token returned once to the phone;
- the initiating private persona ID;
- a start and expiry time;
- a maximum duration of 600 seconds;
- a configured interval clamped to 3–30 seconds;
- accepted-frame counters; and
- declarations that source frames are not retained and visible owner start is required.

Only the token hash is persisted. The token itself is absent from status snapshots and audit events. For an incoming frame with token t , the service requires

$$\text{CTEqual}(\text{SHA256}(t), h_t) = \text{true}, \quad (5)$$

where CTEqual denotes constant-time comparison and h_t is the stored token hash. It additionally requires an active unexpired session and a time since the previous accepted frame greater than or equal to the permitted interval. Invalid, early, stopped, or expired frames are rejected before image analysis.

Stopping the observer is bound to the same private persona. It marks the session inactive and clears the persisted token hash. The start and stop events enter the Fabric audit chain without the bearer token or image content.

7.2 Phone implementation

The controller requests an environment-facing camera stream with audio disabled, shows the live preview, downsizes frames to at most 1,280 pixels on the largest requested dimension, encodes bounded JPEG samples, and sends them to the existing perception endpoint with the `X-Pilot-Observer` session header. The page stops its media tracks when the user stops Observer Mode or leaves the page.

Each accepted frame follows the same local Apple Vision, immediate deletion, screen-fusion, hash, and audit path as a manual capture. Observer Mode changes capture cadence; it does not bypass the normal perception or capability boundary.

7.3 Secure-context limitation

Browser `getUserMedia()` is restricted to secure contexts.[4] The current phone controller is served over token-protected local HTTP for the developer proof. On mobile browsers enforcing the secure-context rule, the UI therefore refuses automatic observation and directs the owner to the existing manual *Take a picture of the TV screen* control. The implementation does not weaken browser security or install an untrusted camera workaround.

A production Observer Mode requires one of:

1. local HTTPS with a certificate chain trusted by the phone; or
2. a signed native Pilot application with explicit camera permission, foreground indicators, operating-system lifecycle handling, and a reviewed privacy declaration.

Until one of these paths exists, repeated automatic camera observation is a backend-complete but browser-constrained capability; manual owner capture is the working developer-preview path.

8 Capability and Interface Boundaries

Table 3: Relevant signed Fabric capabilities.

Capability	Risk	Scope
<code>voice.live.fact_check</code>	Low	Build explicit live context and request evidence-labelled verification.
<code>companion.webos.perception.submit</code>	Low	Submit one explicit bounded phone image for local ephemeral analysis.
<code>companion.screen.context.correct</code>	Low	Correct one allowlisted field of the current structured observation.
<code>companion.screen.observer.control</code>	Medium	Start or stop one visible, short-lived, persona-bound observer session.

Phone operations use a rotating private companion path and a separate CSRF secret. Perception requests are limited to 5 MiB. Ordinary controller requests are limited separately. Observer frames carry the additional bearer token. The server returns permission failures distinctly from malformed input and unexpected processing failures.

The phone response includes a restrictive camera policy scoped to the current origin and disables microphone access on the controller page. The camera belongs to the visual-observer workflow; television audio continues to be handled by the separately governed local voice service.

9 Data Lifecycle and Privacy Model

The system does not infer face identity. A shared television request cannot select a private payment method, account, contact, or calendar automatically. Products, locations, purchases, bookings, and sharing remain within the existing private-phone confirmation model.

Table 4: Retention by data class.

Data	Retention	Persisted derivative
Recent microphone audio	Memory-only; cleared on sleep or stop	Bounded transcript fragments only after explicit contextual request
Phone photograph	Temporary local file deleted after analysis	Hash, byte count, OCR, geometry, labels, and inference
Observer frame	Same immediate deletion as manual capture	Structured perception, fusion result, hash, and session counter
Observer bearer token	Never persisted in raw form	SHA-256 token hash while session remains usable
Provider response	Raw response not retained	Bounded normalized metadata fields and evidence hash
Owner correction	Persisted, persona-bound, maximum 240 characters	Field, value, timestamp, persona ID, and updated evidence hash
Pilot Moment	Persisted signed context record	No protected audio or video

Module	Responsibility
<code>voice.py</code>	Wake phrase, microphone lifecycle, memory-only audio ring, bounded transcript context, and live intents.
<code>live_context.py</code>	Construction and canonical hashing of explicit LiveContext evidence.
<code>moments.py</code>	Rights-aware Moment payload, canonical hash, device signature, and prepared-not-sent state.
<code>apple_vision.swift</code>	Local text recognition, normalized geometry, and image classifications.
<code>perception.py</code>	Input validation, temporary-file lifecycle, structured perception record, and immediate image deletion.
<code>screen_fusion.py</code>	Deterministic source-aware fusion, supported detectors, confidence, conflict state, correction, and evidence hash.
<code>provider_metadata.py</code>	Provider URL normalization and optional bounded official YouTube metadata enrichment.
<code>observer.py</code>	Explicit session lease, persona binding, token hashing, expiry, frame interval enforcement, and stop.
<code>companion.py</code>	Private phone panels, manual capture, Observer controls, camera preview, CSRF, request limits, and frame upload.
<code>runtime.py</code>	Signed capability enforcement, evidence orchestration, provider fusion, audit events, and answer routing.
<code>cli.py</code>	Dashboard, phone snapshot projection, controller services, and runtime status.

10 Implementation Map

11 Verification and Reproducibility

Automated verification covers provider parsing, official-API normalization, entitlement non-claims, Observer token redaction, invalid-token rejection, frame-rate enforcement, persona-bound stop, screen fusion, subtitle and score detection, product confirmation boundaries, game-HUD detection, correction allowlists, companion controls, and the existing Fabric regression suite.

The terminal release evidence is:

- Fabric 0.15.0: 136 passing checks; package SHA-256 c313bdd26da53f1db452e34377d2aef6bd c6dc2510822a64cb54ac439c3d1ed3.
- Fabric 0.16.0: 144 passing checks; package SHA-256 5304d4bdd1beb724fd96f3ee90a14983e7 a67021f51e333b3ac38fbf87aff595.
- Fabric 0.17.0: 149 passing checks; package SHA-256 513eac8a8c0c9e4178837a16103c426615 6341140e609af6430dc920054d49a1.

Fabric 0.15 was field-verified against the paired LG television for permitted state observation and Moment preparation. Fabric 0.16 verified real Apple Vision OCR geometry. The 0.16 and 0.17 implementation and regression runs did not issue live TV control actions, allowing the household game session to continue uninterrupted. At the 0.17 release gate the audit chain was valid, the microphone was physically closed, and Observer Mode was stopped.

12 Honest Capability Boundary

The implemented proof establishes that Pilot can:

- preserve a bounded recent context without retaining raw room audio;
- create a signed current-programme context after explicit request;
- research and display evidence-labelled explanations or fact checks;
- prepare a rights-aware, signed Moment without copying protected media;
- extract OCR text and geometry locally from an explicit phone image;
- recognize a bounded set of subtitle, score, product, location, and game cues;
- fuse independent sources while retaining source and confidence labels;
- let a private persona correct the current observation;
- normalize supported provider identifiers and optionally enrich YouTube metadata; and
- create a visible, expiring, rate-limited Observer authority.

The proof does *not* yet establish that Pilot can:

- see arbitrary protected TV pixels without the owner-provided camera;
- run automatic phone-camera observation over insecure LAN HTTP;
- identify an actor or household member solely from a face;
- guarantee that OCR text is an exact subtitle or factual statement;

- establish a household’s subscription or playback entitlement from a provider URL;
- create or distribute a protected media clip where the provider has not authorized it;
- send a Moment without separate private recipient selection and authorization; or
- provide production-grade continuous perception without native/TLS lifecycle, battery, thermal, privacy, and accessibility testing.

13 Remaining Production Work

The next production-quality steps are:

1. deploy trusted local TLS or a signed native phone application;
2. add a persistent foreground camera indicator, explicit pause, session countdown, battery/thermal adaptation, and operating-system suspension recovery;
3. connect authenticated, persona-specific provider metadata and entitlement adapters without exposing account credentials to the TV;
4. add official provider-native timestamp, Moment, and clip execution where terms and APIs permit it;
5. implement observe–act–observe–verify for supported third-party navigation while preventing automatic legal consent or purchases;
6. improve subtitle, scoreboard, product, and programme identification through provider metadata and temporally stable multi-frame fusion;
7. add spoiler-aware explanation, translation, sports interpretation, and news-claim correction timelines; and
8. complete privacy, accessibility, child-safety, rights, abuse, and regional compliance review before consumer release.

14 Conclusion

Fabric 0.15–0.17 establishes the first governed perception loop for Pilot’s television product. The important architectural result is not a claim that the restricted TV has become an unrestricted camera or media extractor. It is that the mother brain can combine explicit, bounded, independently labelled evidence into useful room-scale intelligence while preserving ownership, privacy, rights, and action boundaries.

The resulting pattern is:

recent context + permitted device state + owner-visible pixels + provider evidence
 $\downarrow$
confidence-labelled understanding $\rightarrow$ private correction $\rightarrow$ governed answer or action.

This is the foundation for live fact checking, contextual explanation, translation, sports intelligence, product discovery, accessibility, education, and rights-aware sharing without requiring a replacement television.

References

- [1] Google for Developers, “YouTube Data API: Videos: list.”
<https://developers.google.com/youtube/v3/docs/videos/list>
- [2] Netflix Help Center, “How to save and share clips from Netflix.”
<https://help.netflix.com/en/node/210664027435620>
- [3] Twitch Developers, “Embedding Video and Clips” and “Everything.”
<https://dev.twitch.tv/docs/embed/video-and-clips/>
<https://dev.twitch.tv/docs/embed/>
- [4] MDN Web Docs, “MediaDevices: getUserMedia() method.”
<https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- [5] LG webOS TV Developer Forum, “Is there an API like in Android TV get screenshot of the screen?”
<https://forum.webostv.developer.lge.com/t/is-there-an-api-like-in-android-tv-get-screenshot-of-the-screen/1419>